

DOCUMENTACIÓN TÉCNICA · v1.0

SmartCommerce

Plataforma de comercio electrónico. Guía técnica de arquitectura, modelo de datos, API y operación para equipos de desarrollo.

Producto

E-commerce (backend + frontend)

Backend

Java 21 · Spring Boot 4 · PostgreSQL 15

Seguridad

Spring Security · JWT (stateless)

Arquitectura

Monolito modular · API REST

Frontend

React · Vite · Tailwind CSS

Infraestructura

Docker / Docker Compose

Autor y responsable técnico

Darius Adrian Benta

Índice de contenidos

1. Introducción

- Propósito del documento
- Qué es SmartCommerce
- Público objetivo

2. Arquitectura

- Vista de alto nivel
- Módulos del sistema
- Arquitectura por capas
- Ciclo de vida de una petición
- Decisiones de diseño

3. Stack tecnológico

4. Estructura del proyecto

5. Modelo de datos

- Conexión
- Relaciones
- Tablas
- Enumerados e integridad

6. Seguridad

- Modelo
- Flujo de autenticación
- Autorización
- Componentes

7. Referencia de la API REST

- Convenciones
- Autenticación, Usuarios, Productos, Categorías
- Carrito, Lista de deseos, Direcciones

8. Módulos del backend

9. Frontend

10. Configuración

11. Puesta en marcha (onboarding)

12. Datos de demostración

13. Manejo de errores

14. Convenciones y flujo de trabajo

15. Roadmap

16. Glosario

1 Introducción

1.1 Propósito del documento

Esta documentación describe SmartCommerce a nivel técnico y funcional, de forma que cualquier persona que se incorpore al equipo pueda comprender el sistema y ser productiva sin necesidad de explicaciones adicionales.

Cubre la arquitectura, el modelo de datos, la seguridad, la referencia completa de la API, la puesta en marcha y las convenciones de trabajo. Está redactada para que también sea comprensible por perfiles no exclusivamente técnicos (p. ej. la parte cliente o de gestión de producto).

1.2 Qué es SmartCommerce

SmartCommerce es una **plataforma de comercio electrónico** de tipo marketplace. Se compone de tres piezas independientes que colaboran entre sí:

Frontend

Interfaz web (SPA) construida en React. Es lo que ve y usa el cliente final. No accede a los datos directamente.

Backend

Aplicación Spring Boot que expone la API REST, aplica las reglas de negocio y persiste la información.

Base de datos

PostgreSQL, donde se almacenan de forma permanente usuarios, productos, carritos, etc.

Funcionalmente cubre: registro e inicio de sesión con roles, catálogo de productos con búsqueda, filtros y paginación, categorías jerárquicas, carrito de la compra, lista de deseos y gestión de direcciones.

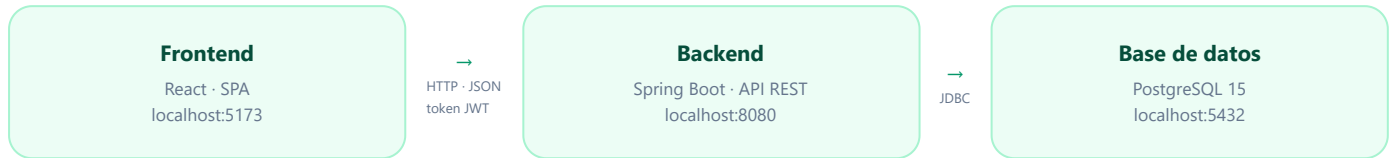
1.3 Público objetivo

Desarrolladores backend, frontend o full-stack que trabajen en el proyecto, así como perfiles de producto o cliente que necesiten entender su alcance y funcionamiento. Se explica el *porqué* de cada decisión para facilitar la incorporación.

2 Arquitectura

2.1 Vista de alto nivel

Tres componentes desacoplados que se comunican mediante HTTP. El frontend nunca accede a la base de datos: todo pasa por la API, y la identidad del usuario viaja en un token de seguridad.



2.2 Módulos del sistema

El backend es un **monolito modular**: una única aplicación desplegable, organizada internamente en módulos por dominio de negocio. Cada módulo es autónomo.

Módulo	Responsabilidad
auth	Registro e inicio de sesión; emisión del token de acceso.
user	Perfil del usuario y administración de usuarios (listar, consultar, desactivar, promover a administrador).
product	Catálogo: alta y edición (administrador), listado público con filtros y paginación, detalle.
productimage	Imágenes asociadas a cada producto.
category	Categorías jerárquicas (con subcategorías).
cart	Carrito de la compra: añadir, actualizar cantidad, eliminar y vaciar, con validación de stock.
wishlist	Lista de deseos (favoritos) por usuario.
address	Direcciones del usuario y dirección predeterminada.
security	Configuración de seguridad, filtro de autenticación y gestión de tokens (transversal).
config / common / exception	Inicialización de datos, utilidades compartidas y manejo global de errores.

2.3 Arquitectura por capas

Dentro de cada módulo, el código se organiza en capas con una responsabilidad única. La información fluye de arriba hacia abajo, y las entidades internas se transforman en objetos de transferencia (DTO) antes de salir por la API.

Controlador — recibe la petición HTTP, valida la entrada y delega. No contiene lógica de negocio.



Servicio — el núcleo: aplica las reglas de negocio (stock, permisos, cálculos).



Repositorio — acceso a datos; lee y escribe en la base de datos.



Entidad — representación de una tabla (uso interno). Se expone al exterior como **DTO**.

2.4 Ciclo de vida de una petición

Ejemplo real: el usuario cambia la cantidad de un producto en su carrito.

- 1 El **frontend** envía la petición a la API con la identidad del usuario en la cabecera de seguridad.
- 2 El **filtro de autenticación** valida el token y determina quién es el usuario.
- 3 La **configuración de seguridad** comprueba que la ruta requiere estar autenticado (se cumple).
- 4 El **controlador** recibe la petición y delega en el servicio.
- 5 El **servicio** valida el stock y actualiza la cantidad dentro de una transacción.
- 6 El **repositorio** persiste el cambio en la base de datos.
- 7 La respuesta devuelve el carrito actualizado (con subtotales y total recalculados) en formato JSON.

2.5 Decisiones de diseño

Decisión	Motivo	Alternativa descartada
Monolito modular	El dominio es un único e-commerce; aporta orden y velocidad sin la complejidad operativa de los microservicios. Los módulos quedan desacoplados y extraíbles a futuro.	Microservicios (sobredimensionado).
Arquitectura por capas	Separa responsabilidades; facilita localizar, probar y mantener el código.	Lógica en los controladores.
PostgreSQL	El dominio es fuertemente relacional y requiere integridad referencial y transacciones.	Base de datos NoSQL.
JWT sin estado	API escalable y desacoplada de una aplicación de página única (SPA); sin sesión en el servidor.	Sesiones en servidor.
DTOs + mapeo automático	Protege el modelo interno, define un contrato de API estable y reduce código repetitivo.	Exponer entidades directamente.
Frontend desacoplado	Permite desarrollar, desplegar y escalar frontend y backend por separado.	Renderizado en servidor.

3 Stack tecnológico

Capa	Tecnología	Función
Backend	Java 21	Lenguaje base (versión con soporte a largo plazo).
	Spring Boot 4	Framework de la aplicación: servidor embebido y autoconfiguración.
	Spring Data JPA / Hibernate	Persistencia orientada a objetos (ORM) sobre PostgreSQL.
	Spring Security + JWT	Autenticación y autorización sin estado.
	Jakarta Validation	Validación declarativa de las peticiones.
	MapStruct · Lombok	Mapeo entidad→DTO y reducción de código repetitivo.
	springdoc-openapi (Swagger)	Documentación interactiva de la API.
Frontend	React + Vite	Interfaz de página única y entorno de desarrollo con recarga en caliente.
	React Router	Navegación sin recargar la página.
	axios	Cliente HTTP con interceptores para el token y los errores.
	Tailwind CSS	Sistema de estilos mediante clases utilitarias.
Datos	PostgreSQL 15	Almacenamiento relacional.
Infra	Docker / Docker Compose · Maven · npm	Ejecución reproducible y gestión de dependencias.

4 Estructura del proyecto

El repositorio es un **monorepo**: backend y frontend conviven para facilitar su orquestación conjunta. Cada módulo del backend sigue el mismo patrón de carpetas.

Ubicación	Contenido
src/main/java/.../	Código del backend, organizado por módulos (auth, user, product, category, cart, wishlist, address, security, config, common, exception).
· cada módulo	Sigue el patrón: <code>controller</code> · <code>service (+impl)</code> · <code>repository</code> · <code>dto</code> · <code>mapper</code> · <code>entity</code> .
src/main/resources/	Configuración de la aplicación (<code>application.properties</code>).
src/test/	Pruebas automatizadas.
frontend/src/	Cliente React, por capas: <code>api</code> · <code>services</code> · <code>context</code> · <code>hooks</code> · <code>components</code> · <code>pages</code> · <code>layouts</code> · <code>routes</code> .
docs/	Documentación del proyecto.
docker-compose.yml	Orquestación de la base de datos.
pom.xml	Dependencias y compilación del backend.

5 Modelo de datos

5.1 Conexión

Parámetro	Valor por defecto
Motor	PostgreSQL 15
Host y puerto	localhost : 5432
Base de datos	smartcommerce
Usuario / contraseña	postgres / postgres
Generación del esquema	Automática a partir de las entidades (Hibernate).

5.2 Relaciones entre entidades

El modelo es relacional. Estas son las relaciones principales:

Entidad	Relación	Entidad	Significado
users	1 : N	addresses	Un usuario tiene varias direcciones.
users	1 : 1	cart	Un usuario tiene un único carrito.
users	1 : N	wishlist_items	Un usuario tiene varios favoritos.
cart	1 : N	cart_items	Un carrito tiene varias líneas.
products	1 : N	cart_items / wishlist_items	Un producto aparece en varias líneas/favoritos.
products	1 : N	product_images	Un producto tiene varias imágenes.
categories	N : 1	products	Cada producto pertenece a una categoría.
categories	1 : N	categories	Jerarquía: una categoría tiene subcategorías (autorreferencia).

5.3 Tablas

users

Columna	Notas
id	Clave primaria
first_name · last_name	Obligatorios
email	Único, obligatorio
password	Cifrada (BCrypt)
phone · birth_date	Opcionales
role	USER o ADMIN
active	Estado de la cuenta
created_at	Fecha de alta

products

Columna	Notas
id	Clave primaria
name	Obligatorio
description · brand	Opcionales
price	Obligatorio
stock	Obligatorio
status	ACTIVE / OUT_OF_STOCK / DISABLED
category_id	Referencia a categoría

categories

Columna	Notas
id	Clave primaria
name	Obligatorio
slug	Único (identificador para URL)
active	Estado
parent_id	Categoría padre (nulo si es raíz)

cart · cart_items

Columna	Notas
cart.user_id	Único (un carrito por usuario)
cart_items.product_id	Producto de la línea
cart_items.quantity	Cantidad

addresses

Columna	Notas
user_id	Propietario
street · city · province · postal_code · country	Obligatorios
address_line2	Opcional
is_default	Dirección predeterminada

wishlist_items · product_images

Columna	Notas
wishlist (user_id, product_id)	Pareja única
product_images.image_url	URL de la imagen

5.4 Enumerados e integridad

Enumerados

Role: USER, ADMIN.

ProductStatus: ACTIVE, OUT_OF_STOCK, DISABLED. Se recalcula según el stock; el "borrado" de producto es lógico (pasa a DISABLED).

Reglas de integridad

Las claves foráneas impiden referencias huérfanas: no se puede borrar un producto referenciado por un carrito o una lista de deseos.

`email` y `slug` son únicos; la pareja usuario+producto en favoritos también.

6 Seguridad

6.1 Modelo de seguridad

Autenticación **sin estado** basada en **tokens JWT**: el servidor no guarda sesiones; cada petición se valida de forma independiente. La autorización se basa en **roles** (USER, ADMIN). Las contraseñas se almacenan cifradas con BCrypt (irreversible).

6.2 Flujo de autenticación

- 1 El usuario inicia sesión con su email y contraseña.
- 2 El backend verifica la contraseña y genera un token firmado con una clave secreta.
- 3 Devuelve el token junto con los datos básicos del usuario (identificador, email y rol).
- 4 El cliente guarda el token y lo adjunta en la cabecera de cada petición posterior.
- 5 El filtro de autenticación valida la firma y la caducidad, e identifica al usuario.
- 6 El controlador ya conoce al usuario autenticado y procesa la petición.

El token **caduca en 1 hora**. Superado ese plazo hay que volver a iniciar sesión. Códigos de seguridad: **401** (falta el token o es inválido) y **403** (autenticado pero sin permiso).

6.3 Autorización

Recurso	Nivel de acceso
Registro e inicio de sesión	Público
Consulta del catálogo (productos)	Público
Documentación de la API (Swagger)	Público
Operaciones de administración (productos, categorías, usuarios)	Solo ADMIN
Carrito, favoritos, direcciones, perfil	Usuario autenticado

6.4 Componentes

Configuración de seguridad

Define las reglas de acceso por ruta, activa la política sin estado, gestiona CORS y registra el filtro de autenticación.

Filtro de autenticación

Se ejecuta en cada petición: extrae y valida el token, y establece la identidad del usuario.

Servicio de tokens

Genera y valida los tokens (firma, extracción del usuario, comprobación de caducidad).

CORS

Autoriza explícitamente el origen del frontend, ya que frontend y backend corren en orígenes distintos.

7 Referencia de la API REST

7.1 Convenciones

Aspecto	Detalle
URL base	<code>http://localhost:8080/api/v1</code>
Formato	JSON en peticiones y respuestas.
Autenticación	Cabecera de tipo "Bearer" con el token en los endpoints protegidos.
Paginación	Parámetros de página, tamaño y orden en el listado de productos.
Documentación viva	Swagger UI en <code>/swagger-ui.html</code> .

Nivel de acceso: público · autenticado · ADMIN.

7.2 Autenticación · Usuarios · Productos · Categorías

POST	<code>/auth/register</code>	público
Registra un usuario y devuelve un token de acceso.		
POST	<code>/auth/login</code>	público
Inicia sesión y devuelve el token junto con los datos del usuario.		
GET	<code>/users/me</code>	autenticado
Perfil del usuario autenticado.		
PUT	<code>/users/me</code>	autenticado
Actualiza el perfil propio.		
GET	<code>/admin/users</code>	ADMIN
Lista de usuarios. Complementado por consulta individual, desactivación y promoción a administrador.		
GET	<code>/products</code>	público
Listado paginado con filtros por nombre, categoría, rango de precio y marca.		
GET	<code>/products/{id}</code>	público
Detalle de un producto.		
POST	<code>/admin/products</code>	ADMIN
Crea un producto. Se complementa con edición, gestión de imágenes y borrado lógico.		
GET	<code>/categories</code>	autenticado
Categorías raíz. Consulta por identificador y por subcategorías disponibles. La gestión (crear, editar, desactivar) es exclusiva de administradores.		

7.3 Carrito · Lista de deseos · Direcciones

GET	<code>/cart</code>	autenticado
Devuelve (o crea) el carrito del usuario, con líneas, subtotales y total.		
POST	<code>/cart/items</code>	autenticado
Añade un producto al carrito, validando stock y disponibilidad.		
PUT	<code>/cart/items/{productId}</code>	autenticado
Fija la cantidad de una línea.		

DELETE /cart/items/{productId}

autenticado

Elimina una línea del carrito. Existe además el vaciado completo.

GET /wishlist

autenticado

Productos favoritos del usuario. Se añaden y quitan por identificador de producto.

GET /addresses

autenticado

Direcciones del usuario. Alta, edición, borrado y marcado de dirección predeterminada.

8 Módulos del backend

product

El listado usa filtros dinámicos combinables (nombre, categoría, precio, marca) sobre una única consulta paginada. El estado del producto se recalcula según el stock.

category

Categorías autorreferenciadas (jerarquía). El identificador para URL se genera automáticamente; la edición valida que no se creen ciclos.

cart

Un carrito por usuario. Añadir suma a la cantidad existente y valida stock; la respuesta incluye subtotales y total calculados.

wishlist

Favoritos con restricción única usuario+producto. El frontend mantiene el estado para mostrar el favorito de forma coherente en todas las vistas.

user

Integra la identidad con el sistema de seguridad. Separa el perfil propio de las operaciones de administración.

config

Inicializadores de datos (usuario administrador y catálogo de demostración), diseñados para no duplicar información.

9 Frontend

Aplicación de página única (SPA) en React que consume la API. Se organiza por capas:

Capa	Contenido
api	Cliente HTTP único con interceptores que añaden el token en cada petición y gestionan la expiración de sesión.
services	Una función por operación de la API, agrupadas por dominio (productos, carrito, categorías, favoritos, autenticación).
context	Estado global: sesión del usuario (persistida entre recargas) y favoritos.
hooks	Lógica reutilizable, como el retardo de la búsqueda para no consultar en cada tecla.
components	Piezas de interfaz reutilizables (tarjeta de producto, control de rango de precio, indicadores de carga, etc.).
pages	Vistas completas: catálogo, carrito, favoritos, detalle de producto, inicio de sesión y registro.
layouts / routes	Estructura común y control de navegación, incluida la protección de las vistas privadas.

Gestión de estado: la interfaz reacciona a los cambios de estado. Valores como el total del carrito son estado *derivado*: se recalculan automáticamente a partir de los datos actuales, sin almacenarse por separado.

10 Configuración

La configuración del backend está centralizada en un único fichero de propiedades.

Parámetro	Descripción
Conexión a la base de datos	URL, usuario y contraseña de PostgreSQL. Deben coincidir con la configuración de Docker.
Gestión del esquema	Hibernate crea o actualiza las tablas según las entidades, sin borrar datos (modo desarrollo).
Clave y caducidad del token	Clave secreta de firma y duración del token (1 hora).
Usuario administrador inicial	Credenciales del administrador creado automáticamente al arrancar.
Pagos y correo	Configuración preparada para integraciones futuras (pasarela de pago y envío de emails).

En un entorno de producción, los datos sensibles (credenciales y clave de firma) deben externalizarse a variables de entorno y no incluirse en el repositorio.

11 Puesta en marcha (onboarding)

Requisitos: Java 21, Docker, Node.js. El sistema se arranca en tres pasos, en orden.

Paso	Acción	Comando
1	Levantar la base de datos (Docker)	<code>docker compose up -d</code>
2	Arrancar el backend → :8080	<code>./mvnw spring-boot:run</code>
3	Arrancar el frontend → :5173	<code>npm install</code> <code>npm run dev</code>

Verificación

Backend operativo: la documentación Swagger responde en `/swagger-ui.html`. El acceso de administrador se crea automáticamente al arrancar.

Añadir un módulo nuevo

Crear el paquete del módulo con sus capas (entidad, repositorio, DTOs, mapeador, servicio y controlador), y proteger sus rutas en la configuración de seguridad si procede.

12 Datos de demostración

Al arrancar, el backend rellena la base de datos de forma automática e **idempotente** (solo crea lo que falta, sin duplicar):

Usuario administrador

Se crea un administrador inicial si no existe, para poder gestionar el catálogo desde el primer momento.

Catálogo de demostración

Se crean 7 categorías y 18 productos de ejemplo, comprobando previamente que no existan.

Gracias a esto, cualquier persona que clone el proyecto —o que reinicie la base de datos desde cero— dispone de un catálogo listo sin intervención manual. Es la base de una demostración "de un solo comando".

13 Manejo de errores

El sistema centraliza el tratamiento de errores y devuelve siempre una respuesta con la misma estructura: marca de tiempo, código de estado y mensaje.

Código	Situación	Ejemplo
400 Petición incorrecta	Validación fallida o regla de negocio incumplida	Falta un campo obligatorio; stock insuficiente.
401 No autorizado	Falta el token o credenciales incorrectas	Inicio de sesión fallido.
403 Prohibido	Autenticado pero sin permiso	Un usuario intenta una acción de administrador.
404 No encontrado	El recurso no existe	Producto, usuario o categoría inexistente.
409 Conflicto	Estado incompatible con la operación	Email ya registrado; borrar la dirección predeterminada.

14 Convenciones y flujo de trabajo

14.1 Ramas

Rama	Uso
main	Versión estable y entregable.
develop	Integración del trabajo en curso.
feature / fix / docs	Ramas de trabajo para funcionalidad, corrección o documentación.

El trabajo se realiza en una rama específica, se abre una **solicitud de fusión (Pull Request)** hacia `develop`, se revisa y se fusiona. Periódicamente, `develop` se promociona a `main`.

14.2 Mensajes de commit

Se sigue el estándar **Conventional Commits**: un tipo, un ámbito opcional y una descripción breve.

Tipo	Significado
feat	Nueva funcionalidad.
fix	Corrección de un error.
docs	Cambios en la documentación.
refactor / chore / test	Reestructuración, tareas de mantenimiento y pruebas.

15 Roadmap

Pedidos

Convertir el carrito en un pedido con su histórico y estados.

Pagos

Integración de una pasarela de pago (base ya preparada).

Notificaciones por correo

Envío de emails transaccionales (base ya preparada).

Contenerización completa

Empaquetar backend y frontend para levantar toda la aplicación con un único comando.

16 Glosario

Término	Definición
API REST	Interfaz basada en HTTP con direcciones (endpoints) para operar sobre los recursos del sistema.
DTO	Objeto de transferencia de datos; entrada/salida de la API, separado de la entidad interna.
Entidad	Representación de una tabla de la base de datos dentro del backend.
ORM	Tecnología que traduce entre objetos del código y filas de las tablas.
JWT	Token firmado que autentica cada petición sin necesidad de sesión en el servidor.
Sin estado (stateless)	El servidor no guarda sesión; cada petición se autentica de forma independiente.
CORS	Permiso que concede el backend para ser llamado desde el origen del frontend.
BCrypt	Algoritmo irreversible para almacenar contraseñas de forma segura.
SPA	Aplicación de página única: cambia de vista sin recargar el navegador.
Idempotente	Operación que puede repetirse sin alterar el resultado (no duplica).

Documento vivo. Esta documentación debe evolucionar con el proyecto: al incorporar un módulo o un endpoint, conviene actualizar las secciones de modelo de datos, API y módulos para mantenerla fiel al sistema.